



API Liga Deportiva

Sports League API

Fase 2: Players y Relaciones

Conceptos Clave

Foreign Keys | Navigation Properties | Enums | Relaciones 1:N
Entity Framework Core Migrations | Validaciones de Negocio

Guía paso a paso para construcción en clase

1. Objetivos de la Fase 2

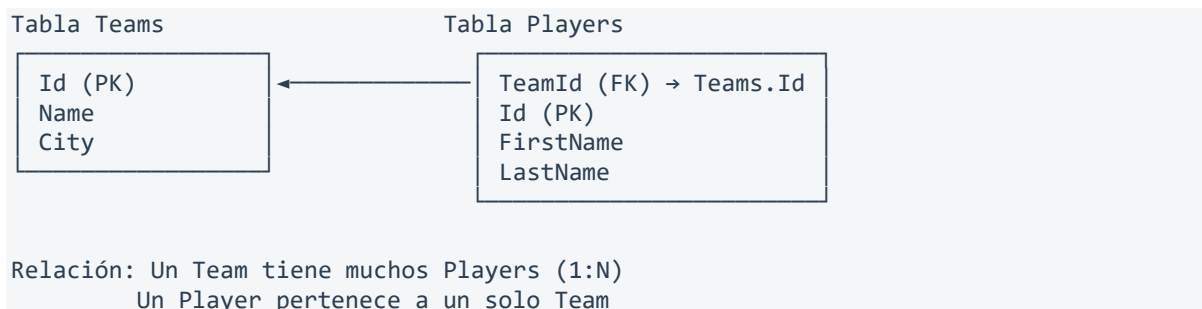
Al finalizar esta fase, los estudiantes serán capaces de:

- Crear entidades con Foreign Keys que referencian a otras tablas
- Implementar Navigation Properties para navegar entre entidades relacionadas
- Definir y usar tipos enumerados (Enums) en las entidades
- Configurar relaciones 1:N con Fluent API en Entity Framework Core
- Reutilizar el patrón Repository genérico para nuevas entidades
- Implementar validaciones de negocio que involucran relaciones entre entidades
- Generar y aplicar nuevas migraciones sobre una base de datos existente

2. Conceptos Nuevos en esta Fase

2.1 Foreign Key (Clave Foránea)

Una Foreign Key es una columna que establece una relación entre dos tablas. En nuestro caso, cada jugador (Player) pertenece a un equipo (Team). La columna TeamId en la tabla Players es una FK que apunta al Id de la tabla Teams.



2.2 Navigation Properties

Son propiedades en las entidades C# que permiten navegar de una entidad a otra sin escribir JOINS manuales. EF Core las resuelve automáticamente:

```
// En Player: navigation property hacia Team
```

```
public Team Team { get; set; } // Acceso: player.Team.Name

// En Team: colección de Players
public ICollection<Player> Players { get; set; } // Acceso: team.Players.Count
```

¿Por qué Navigation Properties?

Sin ellas, para obtener el nombre del equipo de un jugador necesitaríamos hacer dos queries separados. Con Navigation Properties, EF Core puede cargar la información relacionada automáticamente usando `.Include()`. Esto simplifica enormemente el código.

2.3 Enums (Tipos Enumerados)

Los Enums nos permiten definir un conjunto fijo de valores para una propiedad. En lugar de usar strings como "Goalkeeper" o "Defender" (que son propensos a errores de escritura), usamos un enum tipado:

```
// En lugar de: public string Position { get; set; } // "Goalkeeper", "defender",
// "FORWARD"...
// Usamos:      public PlayerPosition Position { get; set; } // Solo valores válidos

// EF Core almacena el enum como int en la BD:
// Goalkeeper = 0, Defender = 1, Midfielder = 2, Forward = 3
```

3. Crear el Enum PlayerPosition

SportsLeague.Domain/Enums/PlayerPosition.cs

```
namespace SportsLeague.Domain.Enums;

public enum PlayerPosition
{
    Goalkeeper = 0,
    Defender = 1,
    Midfielder = 2,
    Forward = 3
}
```



Tip

Asignar valores explícitos (= 0, = 1, etc.) es una buena práctica. Si mañana alguien reordena los elementos del enum, los valores en la base de datos no se corrompen. Además, hace más claro qué número corresponde a cada posición.

4. Crear la Entity Player

SportsLeague.Domain/Entities/Player.cs

```
using SportsLeague.Domain.Enums;

namespace SportsLeague.Domain.Entities;

public class Player : AuditBase
{
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public DateTime BirthDate { get; set; }
    public int Number { get; set; }
    public PlayerPosition Position { get; set; }

    // Foreign Key
    public int TeamId { get; set; }

    // Navigation Property
    public Team Team { get; set; } = null!;
}
```

Anatomía de la relación

TeamId es la Foreign Key (el valor int que se almacena en la columna de la BD). Team es la Navigation Property (el objeto completo que EF Core puede cargar). El = null! le dice al compilador que confiamos en que EF Core inicializará esta propiedad, evitando warnings de nullable.

5. Actualizar la Entity Team

Agregamos la Navigation Property de colección en Team para representar el otro lado de la relación 1:N:

SportsLeague.Domain/Entities/Team.cs (actualizado)

```
namespace SportsLeague.Domain.Entities;
```

```
public class Team : AuditBase
{
    public string Name { get; set; } = string.Empty;
    public string City { get; set; } = string.Empty;
    public string Stadium { get; set; } = string.Empty;
    public string? LogoUrl { get; set; }
    public DateTime FoundedDate { get; set; }

    // Navigation Property - Colección de jugadores
    public ICollection<Player> Players { get; set; } = new List<Player>();
}
```

ICollection vs List vs IEnumerable

Usamos `ICollection<Player>` porque es la interfaz que EF Core espera para Navigation Properties de colección. Permite agregar, remover y contar elementos. La inicializamos como `new List<Player>()` para evitar `NullReferenceException` si accedemos a la colección antes de que EF Core la cargue.

6. Actualizar el DbContext

Agregamos el `DbSet` de `Player` y configuramos la relación con Fluent API:

SportsLeague.DataAccess/Context/LeagueDbContext.cs (actualizado)

```
using Microsoft.EntityFrameworkCore;
using SportsLeague.Domain.Entities;

namespace SportsLeague.DataAccess.Context;

public class LeagueDbContext : DbContext
{
    public LeagueDbContext(DbContextOptions<LeagueDbContext> options)
        : base(options)
    {
    }

    public DbSet<Team> Teams => Set<Team>();
    public DbSet<Player> Players => Set<Player>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // — Team Configuration —
        modelBuilder.Entity<Team>(entity =>
        {

```

```

        entity.HasKey(t => t.Id);
        entity.Property(t => t.Name)
            .IsRequired()
            .HasMaxLength(100);
        entity.Property(t => t.City)
            .IsRequired()
            .HasMaxLength(100);
        entity.Property(t => t.Stadium)
            .HasMaxLength(150);
        entity.Property(t => t.LogoUrl)
            .HasMaxLength(500);
        entity.Property(t => t.CreatedAt)
            .IsRequired();
        entity.Property(t => t.UpdatedAt)
            .IsRequired(false);
        entity.HasIndex(t => t.Name)
            .IsUnique();
    });

    // — Player Configuration —
    modelBuilder.Entity<Player>(entity =>
    {
        entity.HasKey(p => p.Id);
        entity.Property(p => p.FirstName)
            .IsRequired()
            .HasMaxLength(80);
        entity.Property(p => p.LastName)
            .IsRequired()
            .HasMaxLength(80);
        entity.Property(p => p.BirthDate)
            .IsRequired();
        entity.Property(p => p.Number)
            .IsRequired();
        entity.Property(p => p.Position)
            .IsRequired();
        entity.Property(p => p.CreatedAt)
            .IsRequired();
        entity.Property(p => p.UpdatedAt)
            .IsRequired(false);

        // Relación 1:N con Team
        entity.HasOne(p => p.Team)
            .WithMany(t => t.Players)
            .HasForeignKey(p => p.TeamId)
            .OnDelete(DeleteBehavior.Cascade);

        // Índice único compuesto: número de camiseta único por equipo
        entity.HasIndex(p => new { p.TeamId, p.Number })
            .IsUnique();
    });
}

```

 **Enseñanza clave: Fluent API para relaciones**

HasOne(p => p.Team) indica que un Player tiene un Team. WithMany(t => t.Players) indica que un Team tiene muchos Players. HasForeignKey(p => p.TeamId) define cuál es la columna FK. OnDelete(DeleteBehavior.Cascade) significa que si se elimina un Team, se eliminan automáticamente todos sus Players. El índice compuesto (TeamId + Number) garantiza que no haya dos jugadores con el mismo número en el mismo equipo.

7. Definir las Interfaces de Player

7.1 IPlayerRepository

SportsLeague.Domain/Interfaces/Repositories/IPlayerRepository.cs

```
using SportsLeague.Domain.Entities;

namespace SportsLeague.Domain.Interfaces.Repositories;

public interface IPlayerRepository : IGenericRepository<Player>
{
    Task<IEnumerable<Player>> GetByTeamAsync(int teamId);
    Task<Player?> GetByTeamAndNumberAsync(int teamId, int number);
    Task<IEnumerable<Player>> GetAllWithTeamAsync();
    Task<Player?> GetByIdWithTeamAsync(int id);
}
```

Métodos WithTeam

GetAllWithTeamAsync y GetByIdWithTeamAsync son métodos que usan .Include(p => p.Team) para cargar la información del equipo junto con el jugador en una sola query. Sin estos métodos, player.Team sería null (Lazy Loading está deshabilitado por defecto en EF Core).

7.2 IPlayerService

SportsLeague.Domain/Interfaces/Services/IPlayerService.cs

```
using SportsLeague.Domain.Entities;

namespace SportsLeague.Domain.Interfaces.Services;

public interface IPlayerService
{
    Task<IEnumerable<Player>> GetAllAsync();
    Task<Player?> GetByIdAsync(int id);
}
```

```
Task<IEnumerable<Player>> GetByTeamAsync(int teamId);  
Task<Player> CreateAsync(Player player);  
Task UpdateAsync(int id, Player player);  
Task DeleteAsync(int id);  
}
```

8. Implementar el PlayerRepository

SportsLeague.DataAccess/Repositories/PlayerRepository.cs

```
using Microsoft.EntityFrameworkCore;  
using SportsLeague.DataAccess.Context;  
using SportsLeague.Domain.Entities;  
using SportsLeague.Domain.Interfaces.Repositories;  
  
namespace SportsLeague.DataAccess.Repositories;  
  
public class PlayerRepository : GenericRepository<Player>, IPlayerRepository  
{  
    public PlayerRepository(LeagueDbContext context) : base(context)  
    {  
    }  
  
    public async Task<IEnumerable<Player>> GetByTeamAsync(int teamId)  
    {  
        return await _dbSet  
            .Where(p => p.TeamId == teamId)  
            .Include(p => p.Team)  
            .ToListAsync();  
    }  
  
    public async Task<Player?> GetByTeamAndNumberAsync(int teamId, int number)  
    {  
        return await _dbSet  
            .FirstOrDefaultAsync(p => p.TeamId == teamId && p.Number == number);  
    }  
  
    public async Task<IEnumerable<Player>> GetAllWithTeamAsync()  
    {  
        return await _dbSet  
            .Include(p => p.Team)  
            .ToListAsync();  
    }  
  
    public async Task<Player?> GetByIdWithTeamAsync(int id)  
    {  
        return await _dbSet  
            .Include(p => p.Team)  
            .FirstOrDefaultAsync(p => p.Id == id);  
    }  
}
```


💡 Include = Eager Loading

El método `.Include(p => p.Team)` le dice a EF Core que haga un JOIN con la tabla Teams y cargue la información del equipo en la misma query. Sin Include, la propiedad Team sería null. Esto se llama Eager Loading y es la forma recomendada de cargar datos relacionados en EF Core.

9. Implementar el PlayerService

SportsLeague.Domain/Services/PlayerService.cs

```
using Microsoft.Extensions.Logging;
using SportsLeague.Domain.Entities;
using SportsLeague.Domain.Interfaces.Repositories;
using SportsLeague.Domain.Interfaces.Services;

namespace SportsLeague.Domain.Services;

public class PlayerService : IPlayerService
{
    private readonly IPlayerRepository _playerRepository;
    private readonly ITeamRepository _teamRepository;
    private readonly ILogger<PlayerService> _logger;

    public PlayerService(
        IPlayerRepository playerRepository,
        ITeamRepository teamRepository,
        ILogger<PlayerService> logger)
    {
        _playerRepository = playerRepository;
        _teamRepository = teamRepository;
        _logger = logger;
    }

    public async Task<IEnumerable<Player>> GetAllAsync()
    {
        _logger.LogInformation("Retrieving all players");
        return await _playerRepository.GetAllWithTeamAsync();
    }

    public async Task<Player?> GetByIdAsync(int id)
    {
        _logger.LogInformation("Retrieving player with ID: {PlayerId}", id);
        var player = await _playerRepository.GetByIdWithTeamAsync(id);

        if (player == null)
            _logger.LogWarning("Player with ID {PlayerId} not found", id);

        return player;
    }
}
```

```
public async Task<IEnumerable<Player>> GetByTeamAsync(int teamId)
{
    // Validar que el equipo existe
    var teamExists = await _teamRepository.ExistsAsync(teamId);
    if (!teamExists)
    {
        _logger.LogWarning("Team with ID {TeamId} not found", teamId);
        throw new KeyNotFoundException(
            $"No se encontró el equipo con ID {teamId}");
    }

    _logger.LogInformation("Retrieving players for team ID: {TeamId}", teamId);
    return await _playerRepository.GetByTeamAsync(teamId);
}

public async Task<Player> CreateAsync(Player player)
{
    // Validar que el equipo existe
    var teamExists = await _teamRepository.ExistsAsync(player.TeamId);
    if (!teamExists)
    {
        _logger.LogWarning("Team with ID {TeamId} not found", player.TeamId);
        throw new KeyNotFoundException(
            $"No se encontró el equipo con ID {player.TeamId}");
    }

    // Validar número de camiseta único en el equipo
    var existingPlayer = await _playerRepository
        .GetByTeamAndNumberAsync(player.TeamId, player.Number);
    if (existingPlayer != null)
    {
        _logger.LogWarning(
            "Number {Number} already taken in team {TeamId}",
            player.Number, player.TeamId);
        throw new InvalidOperationException(
            $"El número {player.Number} ya está en uso en este equipo");
    }

    _logger.LogInformation(
        "Creating player: {FirstName} {LastName}",
        player.FirstName, player.LastName);
    return await _playerRepository.CreateAsync(player);
}

public async Task UpdateAsync(int id, Player player)
{
    var existingPlayer = await _playerRepository.GetByIdAsync(id);
    if (existingPlayer == null)
    {
        throw new KeyNotFoundException(
            $"No se encontró el jugador con ID {id}");
    }

    // Validar que el nuevo equipo existe
    var teamExists = await _teamRepository.ExistsAsync(player.TeamId);
```

```

        if (!teamExists)
        {
            throw new KeyNotFoundException(
                $"No se encontró el equipo con ID {player.TeamId}");
        }

        // Validar número único (si cambió el número o el equipo)
        if (existingPlayer.Number != player.Number ||
            existingPlayer.TeamId != player.TeamId)
        {
            var conflict = await _playerRepository
                .GetByTeamAndNumberAsync(player.TeamId, player.Number);
            if (conflict != null && conflict.Id != id)
            {
                throw new InvalidOperationException(
                    $"El número {player.Number} ya está en uso en este equipo");
            }
        }

        existingPlayer.FirstName = player.FirstName;
        existingPlayer.LastName = player.LastName;
        existingPlayer.BirthDate = player.BirthDate;
        existingPlayer.Number = player.Number;
        existingPlayer.Position = player.Position;
        existingPlayer.TeamId = player.TeamId;

        _logger.LogInformation("Updating player with ID: {PlayerId}", id);
        await _playerRepository.UpdateAsync(existingPlayer);
    }

    public async Task DeleteAsync(int id)
    {
        var exists = await _playerRepository.ExistsAsync(id);
        if (!exists)
        {
            throw new KeyNotFoundException(
                $"No se encontró el jugador con ID {id}");
        }

        _logger.LogInformation("Deleting player with ID: {PlayerId}", id);
        await _playerRepository.DeleteAsync(id);
    }
}

```

Validaciones cruzadas entre entidades

Notar cómo PlayerService inyecta tanto IPlayerRepository como ITeamRepository. Esto le permite validar que el equipo existe antes de crear o actualizar un jugador. También valida que el número de camiseta no esté duplicado dentro del mismo equipo. La lógica de negocio coordina múltiples repositorios.

10. Crear los DTOs de Player

SportsLeague.API/DTOs/Request/PlayerRequestDTO.cs

```
using SportsLeague.Domain.Enums;

namespace SportsLeague.API.DTOs.Request;

public class PlayerRequestDTO
{
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public DateTime BirthDate { get; set; }
    public int Number { get; set; }
    public PlayerPosition Position { get; set; }
    public int TeamId { get; set; }
}
```

SportsLeague.API/DTOs/Response/PlayerResponseDTO.cs

```
using SportsLeague.Domain.Enums;

namespace SportsLeague.API.DTOs.Response;

public class PlayerResponseDTO
{
    public int Id { get; set; }
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public DateTime BirthDate { get; set; }
    public int Number { get; set; }
    public PlayerPosition Position { get; set; }
    public int TeamId { get; set; }
    public string TeamName { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; }
    public DateTime? UpdatedAt { get; set; }
}
```

💡 TeamName en el ResponseDTO

El ResponseDTO incluye TeamName para que el frontend pueda mostrar el nombre del equipo sin tener que hacer una segunda petición. Este campo no existe en la Entity Player, sino que se mapea desde la Navigation Property player.Team.Name usando AutoMapper.

11. Actualizar AutoMapper

Agregar los mappings de Player al perfil existente:

SportsLeague.API/Mappings/MappingProfile.cs (actualizado)

```
using AutoMapper;
using SportsLeague.API.DTOS.Request;
using SportsLeague.API.DTOS.Response;
using SportsLeague.Domain.Entities;

namespace SportsLeague.API.Mappings;

public class MappingProfile : Profile
{
    public MappingProfile()
    {
        // Team mappings
        CreateMap<TeamRequestDTO, Team>();
        CreateMap<Team, TeamResponseDTO>();

        // Player mappings
        CreateMap<PlayerRequestDTO, Player>();
        CreateMap<Player, PlayerResponseDTO>()
            .ForMember(
                dest => dest.TeamName,
                opt => opt.MapFrom(src => src.Team.Name));
    }
}
```

ForMember con MapFrom

La configuración ForMember le indica a AutoMapper cómo llenar TeamName: tomándolo de src.Team.Name (la Navigation Property). Esto solo funciona si el Team fue cargado con .Include() en el Repository. Si Team es null, AutoMapper asignará un string vacío.

12. Crear el PlayerController

SportsLeague.API/Controllers/PlayerController.cs

```
using AutoMapper;
using Microsoft.AspNetCore.Mvc;
using SportsLeague.API.DTOS.Request;
using SportsLeague.API.DTOS.Response;
using SportsLeague.Domain.Entities;
using SportsLeague.Domain.Interfaces.Services;

namespace SportsLeague.API.Controllers;

[ApiController]
```

```
[Route("api/[controller]")]
public class PlayerController : ControllerBase
{
    private readonly IPlayerService _playerService;
    private readonly IMapper _mapper;
    private readonly ILogger<PlayerController> _logger;

    public PlayerController(
        IPlayerService playerService,
        IMapper mapper,
        ILogger<PlayerController> logger)
    {
        _playerService = playerService;
        _mapper = mapper;
        _logger = logger;
    }

    [HttpGet]
    public async Task<ActionResult<IEnumerable<PlayerResponseDTO>>> GetAll()
    {
        var players = await _playerService.GetAllAsync();
        var playersDto = _mapper.Map<IEnumerable<PlayerResponseDTO>>(players);
        return Ok(playersDto);
    }

    [HttpGet("{id}")]
    public async Task<ActionResult<PlayerResponseDTO>> GetById(int id)
    {
        var player = await _playerService.GetByIdAsync(id);

        if (player == null)
            return NotFound(new { message = $"Jugador con ID {id} no encontrado" });

        var playerDto = _mapper.Map<PlayerResponseDTO>(player);
        return Ok(playerDto);
    }

    [HttpGet("team/{teamId}")]
    public async Task<ActionResult<IEnumerable<PlayerResponseDTO>>> GetByTeam(int
teamId)
    {
        try
        {
            var players = await _playerService.GetByTeamAsync(teamId);
            var playersDto = _mapper.Map<IEnumerable<PlayerResponseDTO>>(players);
            return Ok(playersDto);
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(new { message = ex.Message });
        }
    }

    [HttpPost]
    public async Task<ActionResult<PlayerResponseDTO>> Create(PlayerRequestDTO dto)
    {

```

```
try
{
    var player = _mapper.Map<Player>(dto);
    var createdPlayer = await _playerService.CreateAsync(player);

    // Recargar con Team para el response
    var playerWithTeam = await _playerService.GetByIdAsync(createdPlayer.Id);
    var responseDto = _mapper.Map<PlayerResponseDTO>(playerWithTeam);

    return CreatedAtAction(
        nameof(GetById),
        new { id = responseDto.Id },
        responseDto);
}
catch (KeyNotFoundException ex)
{
    return NotFound(new { message = ex.Message });
}
catch (InvalidOperationException ex)
{
    return Conflict(new { message = ex.Message });
}
}

[HttpPut("{id}")]
public async Task<ActionResult> Update(int id, PlayerRequestDTO dto)
{
    try
    {
        var player = _mapper.Map<Player>(dto);
        await _playerService.UpdateAsync(id, player);
        return NoContent();
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return Conflict(new { message = ex.Message });
    }
}

[HttpDelete("{id}")]
public async Task<ActionResult> Delete(int id)
{
    try
    {
        await _playerService.DeleteAsync(id);
        return NoContent();
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
}
```

```
}
```

💡 ¿Por qué recargamos después del Create?

Después de crear el jugador, necesitamos recargarlo con `GetByIdAsync` para que la `Navigation Property Team` esté cargada y `AutoMapper` pueda mapear `TeamName` correctamente. Sin esto, el response del POST no incluiría el nombre del equipo.

📄 Endpoint GET por equipo

Notar la ruta [`HttpGet("team/{teamId}")`] que genera el endpoint GET `/api/Player/team/1`. Esto permite al frontend obtener todos los jugadores de un equipo específico con una sola llamada.

13. Actualizar Program.cs

Agregar el registro de los nuevos servicios y repositorios:

Agregar después de los registros de Team

```
// — Repositories —
builder.Services.AddScoped(typeof(IGenericRepository<>),
    typeof(GenericRepository<>));
builder.Services.AddScoped<ITeamRepository, TeamRepository>();
builder.Services.AddScoped<IPlayerRepository, PlayerRepository>(); // NUEVO

// — Services —
builder.Services.AddScoped<ITeamService, TeamService>();
builder.Services.AddScoped<IPlayerService, PlayerService>(); // NUEVO
```

No olvidar agregar los usings correspondientes en la parte superior de `Program.cs`:

```
using SportsLeague.Domain.Interfaces.Services;
// Este using ya debería existir, pero verificar que los nuevos tipos se resuelvan
```

14. Crear y Aplicar la Migración

Ejecutar los siguientes comandos desde la raíz de la solución para crear la tabla `Players`:

Crear la migración


```
dotnet ef migrations add AddPlayerEntity \
  --project SportsLeague.DataAccess \
  --startup-project SportsLeague.API
```

Aplicar la migración

```
dotnet ef database update \
  --project SportsLeague.DataAccess \
  --startup-project SportsLeague.API
```

💡 Verificar la migración

Antes de aplicar la migración, se recomienda abrir el archivo generado en la carpeta Migrations y revisar que el código generado es correcto. Debe incluir: CreateTable para Players con la columna TeamId, un índice en TeamId (para optimizar JOINS), el índice único compuesto (TeamId + Number), y la FK con restricción de cascada.

15. Probar la API

Ejecutar la aplicación y probar los nuevos endpoints en Swagger:

```
cd SportsLeague.API
dotnet run
```

Paso 1: Crear un equipo (si no existe)

```
POST /api/Team
{
  "name": "Atlético Nacional",
  "city": "Medellín",
  "stadium": "Atanasio Girardot",
  "logoUrl": "https://example.com/nacional.png",
  "foundedDate": "1947-03-07"
}
```

Paso 2: Crear un jugador

```
POST /api/Player
{
  "firstName": "William",
  "lastName": "Tesillo",
  "birthDate": "1990-02-02",
  "number": 02,
  "position": 2,
  "teamId": 1
}
```

i Position como número

En el JSON enviamos position: 2 que corresponde a Midfielder (según nuestro enum: 0=Goalkeeper, 1=Defender, 2=Midfielder, 3=Forward). Swagger mostrará los valores del enum para facilitar la selección.

Verificaciones esperadas

Acción	Código	Resultado
POST /api/Player	201	Retorna jugador con Id, TeamName incluido
GET /api/Player	200	Lista con jugadores y TeamName de cada uno
GET /api/Player/1	200	Jugador específico con TeamName
GET /api/Player/team/1	200	Todos los jugadores del equipo 1
GET /api/Player/team/999	404	Equipo no encontrado
POST con teamId inexistente	404	Equipo no encontrado
POST con número duplicado en mismo equipo	409	Número ya en uso
POST con mismo número en otro equipo	201	Funciona (número es por equipo)
PUT /api/Player/1	204	Actualización exitosa
DELETE /api/Player/1	204	Eliminación exitosa

💡 Prueba adicional: Cascade Delete

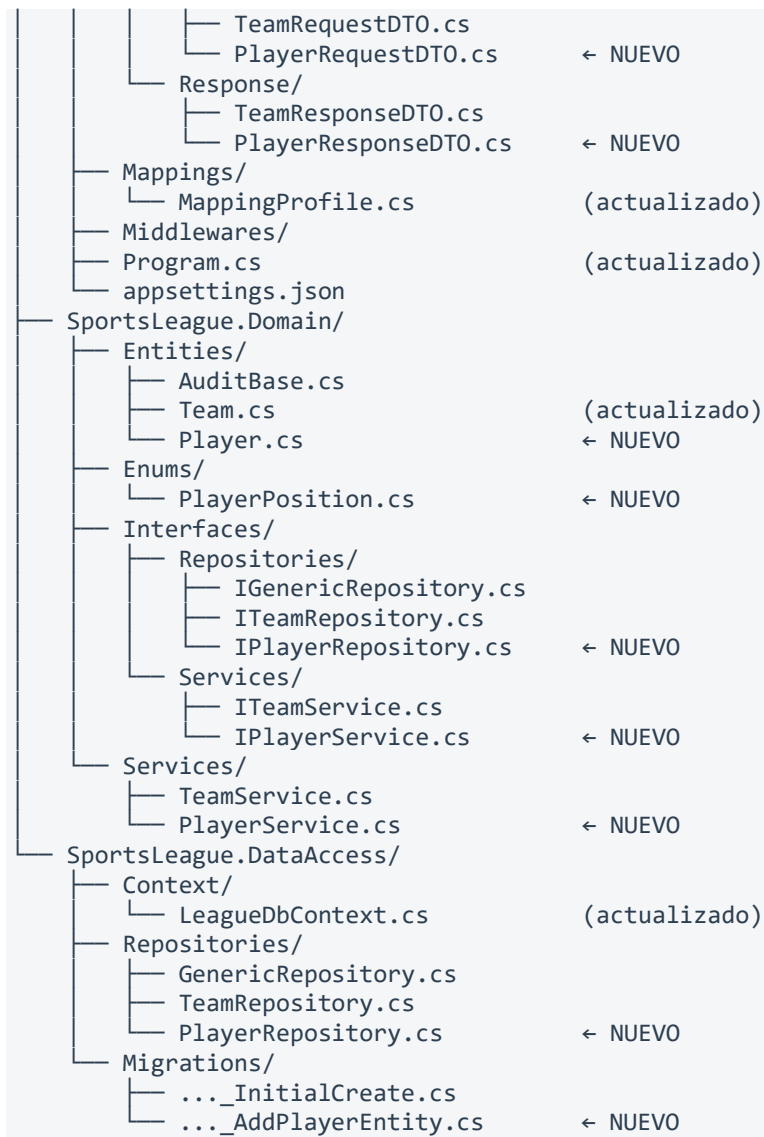
Probar crear un equipo con jugadores y luego eliminar el equipo. Los jugadores deberían eliminarse automáticamente (Cascade). Verificar en la BD o haciendo GET /api/Player.

16. Estructura Final del Proyecto

```

SportsLeague/
├── SportsLeague.sln
├── SportsLeague.API/
│   ├── Controllers/
│   │   ├── TeamController.cs
│   │   └── PlayerController.cs      ← NUEVO
│   ├── DTOs/
│   └── Request/

```



17. Resumen de Conceptos Aprendidos

Concepto	Aplicación en esta Fase
Foreign Key	Player.TeamId referencia a Team.Id
Navigation Property	Player.Team y Team.Players permiten navegar la relación
Enum	PlayerPosition con valores fijos: Goalkeeper, Defender, Midfielder, Forward
Eager Loading (.Include)	Cargar Team junto con Player en una sola query
Fluent API Relaciones	HasOne/WithMany/HasForeignKey/OnDelete en DbContext

Concepto	Aplicación en esta Fase
Índice compuesto único	TeamId + Number para evitar números duplicados por equipo
Cascade Delete	Eliminar Team elimina automáticamente sus Players
Validaciones cruzadas	PlayerService valida existencia de Team antes de crear Player
ForMember en AutoMapper	Mapear Player.Team.Name a PlayerResponseDTO.TeamName
Migraciones incrementales	AddPlayerEntity sobre la BD existente sin perder datos

18. Próxima Fase

En la Fase 3 agregaremos:

- Entity Referee con su CRUD completo
- Entity Tournament con el enum TournamentStatus (Pending, InProgress, Finished)
- Entity TournamentTeam para la relación N:M entre Tournament y Team
- Gestión de estados del torneo (transiciones válidas)
- Inscripción de equipos en torneos con validaciones
- Nuevas migraciones para las 3 tablas